

Задача 1

Исходные данные

Поле $\text{GF}(2^4)$ построено по примитивному полиному $p(x) = x^4 + x + 1$, $\alpha^4 = \alpha + 1$. Длина кода $n = 15 = 2^4 - 1$, конструктивное расстояние $d = 5$ (исправляет до двух ошибок). Порождающий многочлен при $b = 1$:

$$g(x) = \prod_{i=1}^4 (x - \alpha^i).$$

Код имеет параметры $(n, k) = (15, n - d + 1) = (15, 11)$.

Порождающий многочлен

Вычисленный программно многочлен степени 4 имеет коэффициенты:

$$g(x) = x^4 + \alpha^{13}x^3 + \alpha^6x^2 + \alpha^3x + \alpha^{10}.$$

Здесь α – примитивный элемент поля, а коэффициенты представлены в виде степеней α .

Код Python

```
1
2 exp = [0] * 15
3 log = [0] * 16
4 val = 1
5 for i in range(15):
6     exp[i] = val
7     log[val] = i
8     val <= 1
9     if val & 16:
10         val ^= 0x13
11     val &= 0xF
12
13 def gf_add(a, b): return a ^ b
14 def gf_mul(a, b):
15     if a == 0 or b == 0: return 0
16     return exp[(log[a] + log[b]) % 15]
17
18 def gf_poly_mul(poly1, poly2):
19     res = [0] * (len(poly1) + len(poly2) - 1)
20     for i, c1 in enumerate(poly1):
21         for j, c2 in enumerate(poly2):
22             res[i+j] = gf_add(res[i+j], gf_mul(c1, c2))
23     return res
24
25 alphas = [exp[i] for i in [1, 2, 3, 4]]
26 g = [1]
27 for a in alphas:
28     g = gf_poly_mul(g, [1, a])
29
30 print("Generator polynomial g(x) = x^4 + g3 x^3 + g2 x^2 + g1 x + g0")
31 for i, coef in enumerate(reversed(g)):
32     if coef == 0:
33         print(f"g{i} = 0")
34     else:
35         print(f"g{i} = alpha^{log[coef]}")
```

Задача 2

Исходные данные

Поле $\text{GF}(2^4)$ задано примитивным многочленом $p(x) = 1 + x + x^4$. Код (15, 9) с конструктивным расстоянием $d = n - k + 1 = 7$, исправляет до $t = 3$ ошибок. Порождающий многочлен $g(x) = \prod_{i=1}^6 (x - \alpha^i)$.

Принятое слово $v(x) = \sum_{i=0}^{14} v_i x^i$, коэффициенты (степени α):

$$\begin{aligned} v_0 &= \alpha^0, & v_1 &= \alpha^3, & v_2 &= \alpha^{10}, & v_3 &= \alpha^{12}, \\ v_4 &= \alpha^{13}, & v_5 &= \alpha^3, & v_6 &= \alpha^9, & v_7 &= \alpha^5, \\ v_8 &= 1 (\alpha^0), & v_9 &= \alpha^{10}, & v_{10} &= \alpha^8, & v_{11} &= \alpha^1, \\ v_{12} &= \alpha^1, & v_{13} &= \alpha^{13}, & v_{14} &= \alpha^2. \end{aligned}$$

Синдромы

Синдромы $S_j = v(\alpha^j)$, $j = 1, \dots, 6$, вычислены численно:

$$S_1 = \alpha^4, \quad S_2 = \alpha^4, \quad S_3 = \alpha^4, \quad S_4 = \alpha^4, \quad S_5 = \alpha^4, \quad S_6 = \alpha^4.$$

Локатор и значение ошибки

Алгоритм Берлекэмпа–Мессис даёт локатор ошибок:

$$\sigma(x) = 1 + \alpha^0 x + \alpha^0 x^2 + \alpha^0 x^3 = 1 + x + x^2 + x^3.$$

Позиция ошибки: корень $\sigma(\alpha^{-l}) = 0$ при $l = 0$ (индекс 0). Величина ошибки: $e_0 = \alpha^4$.

Вектор ошибок

$$e(x) = \alpha^4.$$

Оошибка присутствует только в позиции x^0 и равна α^4 . Коэффициенты e_i от x^0 до x^{14} :

$$[\alpha^4, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0].$$

Код Python

```
1
2 exp = [0] * 15
3 log = [0] * 16
4 val = 1
5 for i in range(15):
6     exp[i] = val
7     log[val] = i
8     val <<= 1
9     if val & 16:
10         val ^= 0x13
11     val &= 0xF
12
13 def gf_add(a, b): return a ^ b
14 def gf_mul(a, b):
15     if a == 0 or b == 0: return 0
16     return exp[(log[a] + log[b]) % 15]
17 def gf_pow(a, n):
18     if a == 0: return 0 if n != 0 else 1
19     return exp[(log[a] * n) % 15]
20 def gf_inv(a):
21     return exp[(15 - log[a]) % 15]
22
23 v_powers = [0, 3, 10, 12, 13, 3, 9, 5, 0, 10, 8, 1, 1, 13, 2]
24 v = [exp[p] for p in v_powers]
25
26 synd = []
27 for j in range(1, 7):
```

```

28     s = 0
29     for idx, coeff in enumerate(v):
30         s = gf_add(s, gf_mul(coeff, gf_pow(exp[j], idx)))
31     synd.append(s)
32     print(f"S{j} = alpha^{log[s]}")
33
34 S = synd
35 sigma = [1]
36 old_sigma = [1]
37 L = 0
38 m = 1
39 dm = 1
40 for i in range(6):
41     d = 0
42     for j in range(L+1):
43         if j < len(sigma) and sigma[j] != 0:
44             d = gf_add(d, gf_mul(sigma[j], S[i - j]))
45     if d == 0:
46         m += 1
47     else:
48         T = sigma[:]
49         factor = gf_mul(d, gf_inv(dm))
50         while len(sigma) < len(old_sigma) + m:
51             sigma.append(0)
52         for j in range(len(old_sigma)):
53             sigma[j+m] = gf_add(sigma[j+m], gf_mul(factor, old_sigma[j]))
54         if 2*L <= i:
55             L = i + 1 - L
56             old_sigma = T
57             dm = d
58             m = 1
59         else:
60             m += 1
61
62 sigma = sigma[:4]
63 if len(sigma) < 4:
64     sigma += [0]*(4 - len(sigma))
65
66 print(f"sigma(x) = 1 + alpha^{log[sigma[1]]} x + alpha^{log[sigma[2]]} x^2 + alpha^{log[sigma[3]]} x^3")
67
68 error_pos = []
69 for l in range(15):
70     a_inv = exp[(15 - l) % 15]
71     val = 1
72     for j in range(1, 4):
73         val = gf_add(val, gf_mul(sigma[j], gf_pow(a_inv, j)))
74     if val == 0:
75         error_pos.append(l)
76 print(f"Error positions (0-based): {error_pos}")
77
78 S_poly = [1] + synd
79 sigma_poly = sigma
80 omega = [0]*6
81 for i in range(len(S_poly)):
82     for j in range(len(sigma_poly)):
83         if i+j < 6:
84             omega[i+j] = gf_add(omega[i+j], gf_mul(S_poly[i], sigma_poly[j]))
85 omega = omega[:6]
86
87 error_mag = []
88 for l in error_pos:
89     a_inv = exp[(15 - l) % 15]
90     omega_val = 0
91     for i, c in enumerate(omega):
92         omega_val = gf_add(omega_val, gf_mul(c, gf_pow(a_inv, i)))

```

```

93     sigma_deriv = 0
94     for j in range(1, 4, 2):
95         sigma_deriv = gf_add(sigma_deriv, gf_mul(sigma[j], gf_pow(a_inv, j-1)))
96     mag = gf_mul(omega_val, gf_inv(sigma_deriv))
97     error_mag.append(mag)
98     print(f"Error at position {l} magnitude = alpha^{log[mag]}")
99
100 e = [0]*15
101 for pos, mag in zip(error_pos, error_mag):
102     e[pos] = mag
103 e_powers = [log[coef] if coef != 0 else "0" for coef in e]
104 print("Error vector e(x) coefficients from x^0 to x^14 (powers of alpha, 0 for zero):")
105     print(e_powers)

```